PREPARAÇÃO PARA ENTREVISTA TÉCNICA

Fullstack Developer Test

Resumo da implementação, decisões técnicas e roteiro de conversa para apresentar uma aplicação Rails 8 de gestão de usuários.

> **Escopo analisado:** código-fonte, configuração, migrations e suíte RSpec do repositório.
> **Stack:** Ruby 4.0.6 · Rails 8.0.5.1 · PostgreSQL 17 · Docker Compose · Hotwire · Solid Queue/Cable.
> **Data:** 24 de setembro de 2026.

1. Resumo executivo

O projeto é um monólito Rails orientado a servidor para administrar contas de usuários. Ele combina cadastro e perfil do usuário final com um backoffice protegido para administradores. O principal fluxo de maior volume — importação de planilhas — é executado de forma assíncrona, preservando feedback de progresso na interface.

> **Frase de abertura sugerida:** "Escolhi um monólito Rails porque o domínio é coeso: autenticação, gestão de usuários e importação. Isso reduz complexidade operacional, mantém transações e autorização próximas do negócio e ainda permite escalar processamento pesado por workers."

2. Funcionalidades entregues

Autenticação
Cadastro, login, logout, sessão persistida no banco e redefinição de senha por e-mail.

Perfis
Consulta, edição e exclusão da própria conta; suporte a URL de avatar ou upload.

Administração
Dashboard, CRUD de usuários e alteração de papel entre `user` e `admin`.

Importações
Upload de CSV/XLSX, execução em job, isolamento de erros por linha e progresso ao vivo.

3. Arquitetura e responsabilidades

Camada	Responsabilidade na implementação
Models	`User` concentra regras de conta, senha, papéis e avatar; `UserImport` controla arquivo, status e progresso; `Session` registra sessões por usuário.
Controllers	Orquestram requests e usam strong parameters com `params.expect`. Controllers administrativos herdam de `Admin::BaseController`.
Concern	`Authentication` centraliza restauração, criação e término de sessão, além de redirects de autenticação.
Job	`UserImportJob` abre CSV/XLSX com Roo, cria usuários e persiste resultado incrementalmente.
Views/UX	ERB + Tailwind, Turbo Streams para atualizações e Stimulus para feedback imediato de validade de formulários.
Infraestrutura	Compose separa `web`, `worker` e PostgreSQL; Solid Queue e Solid Cable usam banco, evitando Redis.

Modelo de dados

`users` possui nome, e-mail único, `password_digest`, papel e URL de avatar. `sessions` pertence a um usuário e registra IP/User-Agent. `user_imports` pertence ao administrador criador e armazena status, totais e `error_details` em JSONB. Active Storage mantém os metadados de arquivos e relacionamentos.

4. Fluxos que valem explicar

Login e autorização

1. `User.authenticate_by` valida credenciais protegidas por BCrypt (`has_secure_password`).
2. Uma linha em `sessions` é criada e seu id é salvo em cookie assinado, HTTP-only, permanente e SameSite=Lax.
3. Em cada request, o concern recupera a sessão pelo cookie e popula `Current.session`; `Current.user` deriva dela.
4. `Admin::BaseController` aplica a barreira de papel no servidor. O front-end apenas reflete a permissão, não a concede.

O benefício dessa modelagem é poder revogar sessões no banco e manter metadados de acesso. A contrapartida é uma consulta adicional por request, aceitável para este escopo e passível de otimização posterior com expiração/índices/cache.

Importação assíncrona

1. O admin envia CSV ou XLSX; o model valida presença, extensão, MIME type e tamanho máximo de 10 MB.
2. O controller persiste `UserImport` como pendente e enfileira `UserImportJob`.
3. O job valida os headers `full_name` e `email`, lê as linhas com Roo e cria usuários com senha aleatória.
4. Falhas de validação de uma linha são acumuladas, mas não interrompem as linhas válidas. Contadores e erros são atualizados durante a execução.
5. Callbacks transmitem Turbo Stream para a página de importação; falhas estruturais marcam a importação como `failed` e relançam o erro para observabilidade/retry do job.

> **Ponto importante para falar:** contas importadas recebem senha aleatória; portanto o fluxo esperado é usar “esqueci minha senha” antes do primeiro login. Isso evita distribuir senhas em planilha ou em texto claro.

Upload de avatar

O usuário pode anexar JPEG, PNG ou WebP de até 5 MB com Active Storage, ou informar URL HTTP/HTTPS. A validação de negócio acontece no model e `avatar_source` dá prioridade ao arquivo anexado. Assim a view não precisa conhecer a origem do avatar.

5. Segurança e integridade

- Senhas armazenadas como hash BCrypt; nunca há persistência de senha em texto claro.
- CSRF permanece ativo; cookies de sessão são assinados, HTTP-only e SameSite=Lax.
- Login tem rate limit: 10 tentativas em 3 minutos.
- E-mail é normalizado (trim + lowercase), validado e protegido por índice único no banco — não apenas por validação de aplicação.
- Parâmetros são explicitamente permitidos; durante cadastro público o papel é forçado para `user`, impedindo elevação de privilégio.
- Admin não pode excluir a própria conta nem alterar seu próprio papel, protegendo o acesso administrativo atual.
- Validações restringem MIME, extensão e tamanho dos uploads. As views ERB escapam conteúdo por padrão.

Há testes específicos para payload de SQL injection, escape de XSS e configuração de CSRF. A análise estática prevista inclui Brakeman e RuboCop, além de auditoria do import map.

6. Qualidade e testes

A suíte usa RSpec, Factory Bot, Capybara/Selenium e SimpleCov com meta de 90% de cobertura de linhas. A separação é adequada: models para validações, requests para os contratos HTTP e autorização, job para importação, e system spec para a jornada de cadastro no navegador.

Exemplos já cobertos	Por que importa
Normalização e validações do usuário; avatar inválido	Protege invariantes do domínio perto da origem.
CRUD admin, bloqueio de autoalteração e negação ao usuário comum	Valida o limite de autorização por request.
Importação válida/linha inválida e headers ausentes	Confirma processamento parcial e comportamento de erro.
Cadastro via browser	Garante a experiência fim a fim, incluindo formulário e redirect.

7. Melhorias honestas para mencionar

- Adicionar paginação, filtros e busca ao índice administrativo antes de crescer a base de usuários.
- Fazer uma revisão de segurança mais profunda para downloads/URLs remotas de avatar (por exemplo, política de host e mitigação de SSRF se o servidor buscar essas URLs).
- Limitar tamanho/número de linhas da planilha e criar métricas/alertas do job; para grandes volumes, usar inserção em lote e estratégia de idempotência.
- Definir política de expiração/limpeza de sessões e, se necessário, uma tela de dispositivos conectados.
- Adicionar testes de autorização para todos os endpoints e testar os broadcasts Turbo de modo mais direto.

8. Roteiro de 90 segundos

“A aplicação é um monólito Rails 8 para gestão de usuários. Priorizei um desenho simples de operar: PostgreSQL sustenta os dados e também Solid Queue/Solid Cable, então não há dependência de Redis. O fluxo público permite cadastro e gestão do próprio perfil; a área administrativa é protegida no servidor por role-based access control e oferece dashboard, CRUD e importação de planilhas.

Na autenticação, uso BCrypt via `has_secure_password` e sessões persistidas no banco, referenciadas por cookie assinado e HTTP-only. Isso torna revogação e auditoria de sessão mais diretas. Para imports, o request só valida, persiste e enfileira; o worker lê CSV/XLSX, trata erros por linha e transmite progresso para a UI. Cobri regras de domínio, requests de segurança e autorização, job e um fluxo de navegador. Como próximos passos, eu investiria em paginação, proteção adicional para URLs externas e escalabilidade/idempotência de imports.”

Documento gerado a partir da revisão estática deste repositório. Os comandos de verificação documentados no projeto são `docker compose run --rm web bundle exec rspec`, `bin/rubocop` e `bin/brakeman --no-pager`.